PROJECT PHASE 1 - LLM X-RAY

Interactive Visualization of LLM Inference

1. Project Overview

Build a local LLM application that accepts a text prompt, generates a response using a Hugging Face model, and visually shows what happens inside the model during inference.

Core Flow

```
Input → Tokenization → Token IDs → Embeddings → Transformer Layers →
Attention → Logits → Probabilities → Next Token → Response
```

The application should work on a personal laptop without requiring an external LLM API.

2. Model

Use:

- Qwen/Qwen2.5-1.5B-Instruct
- Hugging Face: Qwen2.5-1.5B-Instruct - https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct

Why this model:

- 1.54B parameters
- 28 Transformer layers
- Suitable for local inference
- Instruction-tuned
- Supports Hugging Face Transformers
- Apache 2.0 license

3. Technology Stack

- Python
- PyTorch
- Hugging Face Transformers
- Streamlit — interface
- Plotly — visualizations
- Scikit-learn — PCA for embeddings
- NumPy

4. Implementation Steps

Step 1 — Setup

Create the project:

```
LLM-XRay/
├── app.py
├── model.py
├── tokenizer.py
├── visualization.py
└── requirements.txt
```

Install:

```
pip install torch transformers streamlit plotly scikit-learn numpy
```

Step 2 — Load the Model

Load:

```
Qwen/Qwen2.5-1.5B-Instruct
```

Use Hugging Face:

- AutoTokenizer
- AutoModelForCausalLM

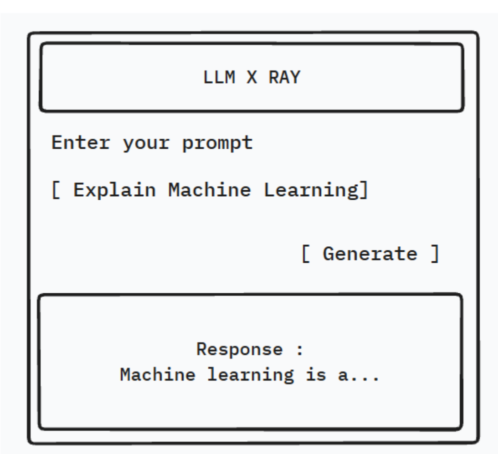
First verify that:

```
Input → Model → Generated Response
```

works locally.

Step 3 — Build Chat Interface

Create a simple interface:



Step 4 — Add Tokenization

After entering the prompt, display:

```
Input
 ↓
Tokenizer
 ↓
Tokens
 ↓
Token IDs
```

Example:

```
Input:
What is AI?

Tokens:
What | is | AI | ?

Token IDs:
XXXX | XXXX | XXXX | XXXX
```

Step 5 — Add Embedding Visualization

Extract the input embeddings.

Display:

```
Token
↓
Token ID
↓
Embedding Vector
```

Show:

- embedding dimension
- vector values
- vector statistics

Use PCA to create a 2D embedding visualization.

Step 6 — Add Transformer Visualization

Display the model architecture:

```
Embeddings
   ↓
Layer 1
   ↓
Layer 2
   ↓
...
   ↓
Layer 28
   ↓
Output
```

Allow the user to select a Transformer layer.

Step 7 — Add Attention Visualization

Extract attention values using the model's attention outputs.

Display them as a heatmap:

```
         What   is   AI   ?
What      █    ░    ░    ░
is       ░    █    ░    ░
AI       ░    ░    █    ░
?        ░    ░    ░    █
```

Add controls:

```
Layer: [12]
Head:  [4]
```

Step 8 — Add Hidden States

Extract hidden states from different Transformer layers.

Allow selection:

- Layer 1
- Layer 10
- Layer 20
- Layer 28

Optionally visualize them using PCA.

Step 9 — Add Logits and Probabilities

After the Transformer:

```
Hidden State
    ↓
LM Head
    ↓
Logits
    ↓
Softmax
    ↓
Probabilities
```

Display the top predictions:

```
Token      Probability

"is"        32%
"was"        18%
"can"        12%
"will"        8%
```

Use a bar chart.

Step 10 — Visualize Token Generation

Show how the response is generated:

```
Prompt
  ↓
Token 1
  ↓
Token 2
  ↓
Token 3
  ↓
Token 4
  ↓
...
  ↓
Final Response
```

Example:

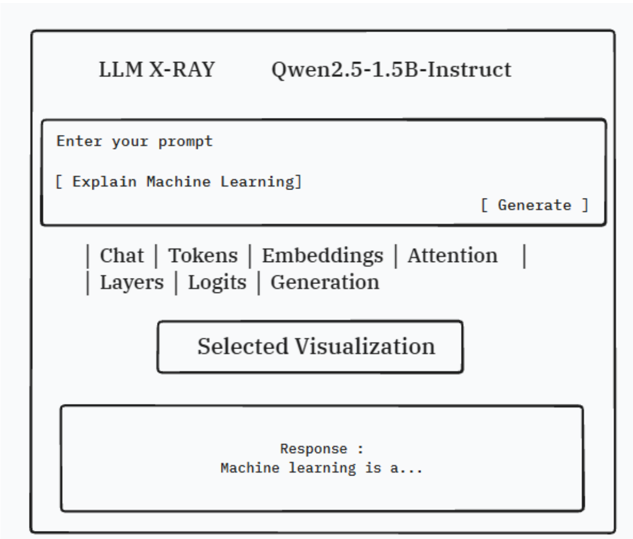
```
What is AI?

→ AI
→ AI is
→ AI is a
→ AI is a field
```

→ AI is a field of
→ ...

5. Final Interface

The application should contain:



6. Optional Controls

Add:

- Temperature
- Top-K
- Top-P
- Maximum new tokens
- Transformer layer
- Attention head

This allows experimentation with how generation changes.

7. Expected Output

The final system should demonstrate:

Prompt
↓
Tokenization
↓
Token IDs
↓
Embeddings

```
↓
Transformer Layers
↓
Attention
↓
Hidden States
↓
Logits
↓
Probability Distribution
↓
Next-Token Prediction
↓
Final LLM Response
```

8. Main Objective

The goal is not to train an LLM from scratch.

The goal is to use a real Hugging Face LLM locally and build an X-Ray/visualization layer around the inference process, making the internal operations of the LLM observable and understandable.

Final Deliverable

A locally running web application where a user can:

Enter a prompt → Generate a response → Inspect tokenization → Explore embeddings → View attention → Inspect logits/probabilities → Observe token-by-token generation.

Please reach out if you have a doubt:
Mr. Yashwanth V S Devang
+91 9964778336

It's time to become much STRONGER